



# CS/CE 224/272 - Object Oriented Programming and Design Methodology

Nadia Nasir



# Functions

# Function Declarations and Definitions

*Functions can be declared?*

Function definitions should come before defining **main()**.

Otherwise, compiler complains.

```
1 | #include <iostream>
2 |
3 | int main()
4 | {
5 |     std::cout << "The sum of 3 and 4 is: " << add(3, 4) << '\n';
6 |     return 0;
7 | }
8 |
9 | int add(int x, int y)
10 | {
11 |     return x + y;
12 | }
```

```
PS C:\Users\mohammad.salman\OneDrive - Habib University\Documents\Course Stuff\Fall 2023\Lectures\Week 4\codes> g++ -pedantic-errors -Wall -Werror -Wextra -Wconversion -Wsign-conversion -std=c++14 .\2a-functions.cpp
```

```
.\2a-functions.cpp: In function 'int main()':
.\2a-functions.cpp:5:47: error: 'add' was not declared in this scope
    std::cout << "The sum of 3 and 4 is: " << add(3, 4) << '\n';
                                              ^~~
```

```
.\2a-functions.cpp:5:47: note: suggested alternative: 'rand'
    std::cout << "The sum of 3 and 4 is: " << add(3, 4) << '\n';
                                              ^~~
                                              rand
```

```
PS C:\Users\mohammad.salman\OneDrive - Habib University\Documents\Course Stuff\Fall 2023\Lect
```

## What's happening?

- Compilers compile the contents of code files **sequentially**.
- So the function definition hasn't been seen before the function is called.
  - Call is in **main()**.
  - Definition is after **main()**.
- Therefore, an error is raised.
- That is why we have been keeping the function definitions **before** those functions are called.

## Concerns with this...

- However, in larger programs, performing this rearrangement, and being mindful of it, can be tedious.
- Additionally, we often use multiple files in larger projects.

## Make use of **function declarations (or function prototypes)**!

Function declarations are used to tell the compiler about the existence of some function that has been defined in a different code file (or later on in the same code file).

```
1  #include <iostream>
2
3  // function declaration
4  int add(int x, int y);
5
6
7  int main()
8  {
9      // this now works
10     std::cout << "The sum of 3 and 4 is: " << add(3, 4) << '\n';
11
12     return 0;
13 }
14
15
16 // function definition
17 int add(int x, int y)
18 {
19     return x + y;
20 }
```

```
PS C:\Users\mohammad.salman\On
ures\Week 4\codes> .\a.exe
The sum of 3 and 4 is: 7
PS C:\Users\mohammad.salman\On
```

# Function Declaration (or Function Prototype)

- A **function declaration** tells the compiler that a function exists.
  - And we will provide the definition somewhere else.
  - *And you must! Otherwise **errors** (there is an exception).*
- This puts the compiler at ease.
- So now, you could have a function definition that is present,
  - after that function is called (in our e.g., after **main()**), or,
  - even in some separate file.

# Function Declaration (or Function Prototype)

- The function declaration consists of the following.
  - Function's return type
  - Function's name, and
  - Function's parameter types
  - A semicolon at the very end.

```
int add(int x, int y);
```

- The function body **is not** included in the declaration.
- This idea is referred to as **forward declaration**.
  - The compiler is made aware of the existence of a function before defining it.



```

1  #include <iostream>
2
3  // function declaration
4  int add(int x, int y);
5
6
7  int main()
8  {
9      // this now works
10     std::cout << "The sum of 3 and 4 is: " << add(3, 4) << '\n';
11
12     return 0;
13 }
14
15
16 // // function definition
17 // int add(int x, int y)
18 // {
19 //     return x + y;
20 // }

```

What if you make a call to a function that was declared, but forgot to provide its definition?

You get a **linker error!** (we'll hopefully see what a linker is later on). The definition for **add(int, int)** can't be found.

If you don't call the declared function, however, then no error comes up.

```

PS C:\Users\mohammad.salman\OneDrive - Habib University\Documents\Course Stuff\Fall 2023\Lect
ures\Week 4\codes> g++ -pedantic-errors -Wall -Werror -Wextra -Wconversion -Wsign-conversion
-std=c++14 .\rough.cpp

```

```

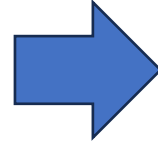
C:\Users\MOHAMM~1\SAL\AppData\Local\Temp\ccRigOpM.o:rough.cpp:(.text+0x34): undefined referen
ce to `add(int, int)'
collect2.exe: error: ld returned 1 exit status

```

# Function Overloading

*Functions can have the same name...*

```
3  int addInteger(int x, int y)
4  {
5      return x + y;
6  }
7
8  double addDouble(double x, double y)
9  {
10     return x + y;
11 }
```



```
3  int add(int x, int y)
4  {
5      return x + y;
6  }
7
8  double add(double x, double y)
9  {
10     return x + y;
11 }
```

*You may need **more functions** that do a very similar task.*

*The problem is that you will have to come up with a **consistent naming scheme**, which can be a pain.*

***Function overloading** allows you to use the same function name.*

*There must be some **uniqueness**, so that the compiler could differentiate between the functions with same names.*

### Note

*For function overloading, the name becomes the same. So the compiler uses other parts to distinguish b/w overloads.*

# Function Overloading

- **How to ensure uniqueness** so that the compiler could differentiate?
- **Function's Type Signature (or Signature)**
  - *Name of the function.*
  - *Number of parameters.*
  - *Data types of parameters.*
  - *One more thing (don't worry for now).*
- ***Return type*** of a function **is not** part of the function signature.

***Return type*** of a function **is not** part of the function signature.

```
26 int foo()  
27 {  
28     return 1;  
29 }  
30  
31  
32 double foo()  
33 {  
34     return 1.1;  
35 }  
36  
37  
38 int main()  
39 {  
40     foo();  
41  
42     return 0;  
43 }
```

```
es\Week 4\codes> g++ -pedantic-errors -Wall -Werror -Wextra -Wconversion -Wsign-conversion -std=c++14 .\2a-functions.cpp  
.\2a-functions.cpp:32:8: error: ambiguating new declaration of 'double foo()' [C++14]  
    double foo()  
           ^~~~  
.\2a-functions.cpp:26:5: note: old declaration 'int foo()' [C++14]  
    int foo()  
       ^~~~
```

*It isn't clear which function should the compiler call.*

*In this case, you might just want to have different function names.*

```
int fooInt()  
double fooDouble()
```

```
3  int add(int x, int y)
4  {
5      return x + y;
6  }
7
8  int add(int x, int y, int z)
9  {
10     return x + y + z;
11 }
```

*Varying the **number of parameters**.*

```
14  double add(int x, double y)
15  {
16      return x + y;
17  }
18
19
20  double add(double x, int y)
21  {
22      return x + y;
23 }
```

*Varying the **list of parameter types**.*

# Default Arguments

# Default Arguments

*A default value of 10 has been set for y.*

```
4 void print(int x, int y=10) // 10 is the default argument
5 {
6     std::cout << "x: " << x << '\n';
7     std::cout << "y: " << y << '\n';
8 }
```

```
11 int main()
12 {
13     print(5, 17); // x: 5, y: 17
14
15     print(5);     // print(5, 10)
16                 // x: 5, y: 10
17
18     return 0;
19 }
```



```
4  ✓ void print(int x, int y=10) // 10 is the default argument
5      {
6          std::cout << "x: " << x << '\n';
7          std::cout << "y: " << y << '\n';
8      }
```

- In order to call this function, two arguments could be provided.
- However, I may choose to **not** pass a value for **y**, since a default value has been specified.
  - So I can pass one value for **x** only.
- If I don't pass a value for **y**, it will have a **value of 10 (value of default argument)**.

```
23 void print( int x, int y{10} )
24 {
25     std::cout << "x: " << x << '\n';
26     std::cout << "y: " << y << '\n';
27 }
28
```

### NOTE ON SYNTAX

*Braces will not work for default arguments.*

*Have to use*  
**int y = 10**

# Another (Important) Syntactical Limitation

If a parameter is given a default argument...

... all subsequent parameters (to the right) **must** also be given default arguments.

*y must also have a default argument.*  
**OR**  
*y should be on the left of x.*

```
23 void print(int x=10, int y) // NOT ALLOWED
24 {
25     // pass
26 }
```

## What will the following function call be interpreted as?

```
3  void print(int x=10, int y)
4  {
5      std::cout << x << ' ' << y << '\n';
6  }
7
8
9  int main()
10 {
11     print(5);
12     return 0;
13 }
```

C++ does **not** support the following syntax:

**print(, 5);**

So there is ambiguity whether this **5** is for **x** or **y**.

```
.\2a-functions.cpp:3:6: error: default argument missing for parameter 2 of 'void print(int, int )'
void print(int x=10, int y)
      ^~~~~
```

## Self Study

### *Things to keep in mind:*

1. In a function call, any **explicitly provided arguments** must be the leftmost arguments (arguments with defaults cannot be skipped).

```
1 void print(std::string_view sv="Hello", double d=10.0);
2
3 int main()
4 {
5     print();           // okay: both arguments defaulted
6     print("Macaroni"); // okay: d defaults to 10.0
7     print(20.0);       // error: does not match above function (cannot skip argument for sv)
8
9     return 0;
10 }
```

## Self Study

2. If a parameter is given a default argument, all subsequent parameters (to the right) must also be given default arguments.

The following is not allowed:

```
1 | void print(int x=10, int y); // not allowed
```

### Rule

If a parameter is given a default argument, all subsequent parameters (to the right) must also be given default arguments.

## **Self Study**

3. If more than one parameter has a default argument, the leftmost parameter should be the one most likely to be explicitly set by the user.

*What will be the output of the following program?*

Default arguments  
with function  
overloading

```
4 void foo(int x)
5 {
6     std::cout << x << '\n';
7 }
8
9 void foo(int x, int y=10)
10 {
11     std::cout << x << '\n';
12     std::cout << y << '\n';
13 }
```

```
16 int main()
17 {
18     foo(5);
19     return 0;
20 }
```

```
.\2a-functions.cpp: In function 'int main()':
.\2a-functions.cpp:18:10: error: call of overloaded 'foo(int)' is ambiguous
    foo(5);
    ^
.\2a-functions.cpp:4:6: note: candidate: 'void foo(int)'
    void foo(int x)
    ^~~
.\2a-functions.cpp:9:6: note: candidate: 'void foo(int, int)'
    void foo(int x, int y=10)
    ^~~
```



# Default Arguments

- The following will not work.

```
1 | #include <iostream>
2 |
3 | void print(int x, int y=4); // forward declaration
4 |
5 | void print(int x, int y=4) // error: redefinition of default argument
6 | {
7 |     std::cout << "x: " << x << '\n';
8 |     std::cout << "y: " << y << '\n';
9 | }
```

in foo.h:

```
1 #ifndef FOO_H
2 #define FOO_H
3 void print(int x, int y=4);
4 #endif
```

in main.cpp:

```
1 #include "foo.h"
2 #include <iostream>
3
4 void print(int x, int y)
5 {
6     std::cout << "x: " << x << '\n';
7     std::cout << "y: " << y << '\n';
8 }
9
10 int main()
11 {
12     print(5);
13
14     return 0;
15 }
```

Put the default argument in either prototype or the definition.

### **Best practice**

If the function has a forward declaration (especially one in a header file), put the default argument there. Otherwise, put the default argument in the function definition.